

Plano de Execução — Réplica Adaptada do W3Schools (CopyKate)

Documento 1 de 2. O documento 2 (`prompt-copykate-w3schools.md`) é o prompt pronto a correr/colar num LLM ou swarm.

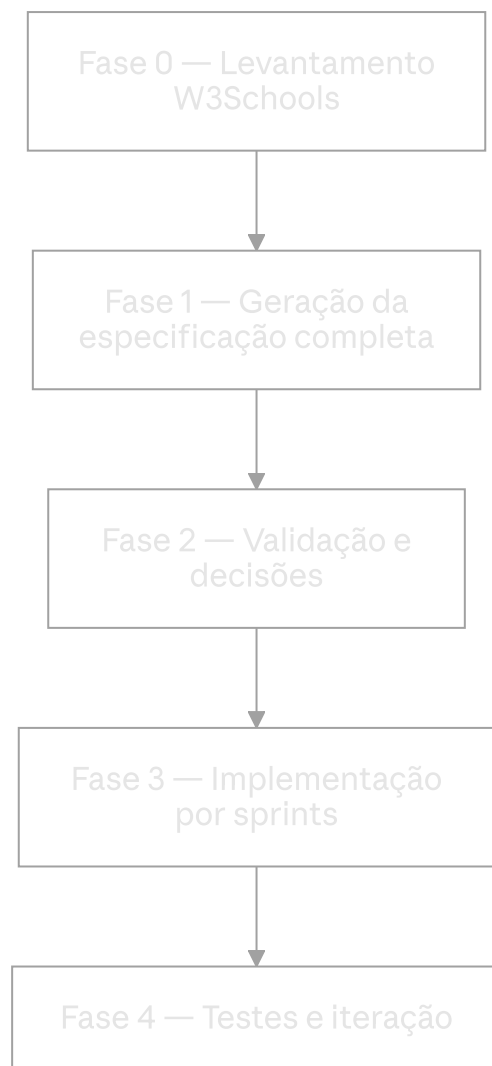
0. Enquadramento

Pediste dois entregáveis: um **plano** e um **prompt**, para estudar/estruturar o W3Schools e produzir uma cópia adaptada e offline com as funcionalidades do projeto CopyKate (IDE embutido, grafo de progresso A1, orquestrador A2, buddy CopyKate, RAG, gamificação, anti-copy), sem login, com rigor no mapa do site, nas funcionalidades e no funcionamento provável de backend.

O teu documento anexado já era um rascunho muito completo. Em vez de o reescrever do zero, fiz três coisas:

1. Fui **diretamente ao W3Schools** confirmar a estrutura real e atual (não assumida de memória) — secção 3.
2. **Mapeei cada peça** dela para o equivalente adaptado no CopyKate — secção 4.
3. **Apertei** os pontos que pediste explicitamente com mais rigor: mapa do site, funcionalidades, hipótese de backend, restrição de login, IDE, layout/HTML+CSS — tudo isso está no ficheiro 2.

1. Metodologia — 4 fases



- **Fase 0 (concluída nesta sessão):** varredura ao W3Schools real. Ver secção 3.
- **Fase 1:** correr o prompt (ficheiro 2) — produz sitemap completo, funcionalidades detalhadas, arquitetura backend, schema de BD, wireframes + HTML/CSS, pseudocódigo dos 3 motores, pipeline RAG, sprints e critérios de aceitação, plano de testes.
- **Fase 2:** revê o output gerado face às decisões em aberto (secção 6) e ajusta.
- **Fase 3:** implementação seguindo o roadmap gerado na Fase 1.
- **Fase 4:** testes automatizados + usabilidade, segundo os critérios definidos na Fase 1.

2. Nota sobre o âmbito desta sessão

Este plano e o prompt anexo **não** geram ainda a especificação completa (sitemap final, schema de BD final, pseudocódigo, etc.) — isso é o trabalho da Fase 1, propositadamente separado porque é um entregável enorme por si só. O que aqui está é o que precisas para arrancar essa fase com uma base real e sem ambiguidades.

3. Varredura ao W3Schools — resultado confirmado

Fui à fonte (w3schools.com — homepage, páginas de tutorial, editor de código, Spaces) em vez de assumir de memória. Resumo do confirmado:

3.1 Navegação de topo e catálogo de tutoriais

Nav principal: **Tutorials** · **References** · **Exercises** · **Certificates**, com um segundo nível de serviços (Spaces, Get Certified, Upgrade/Plus, Academy, Practice).

O catálogo de tutoriais tem hoje **mais de 45 tracks** — bem mais do que os "5 grandes" (HTML/CSS/JS/Python/SQL) que normalmente se assume. Confirmei, entre outros: HTML, CSS, JavaScript, SQL, Python, Java, PHP, C, C++, C#, Bootstrap, React, MySQL, jQuery, Excel, XML, Django, NumPy, Pandas, Node.js, DSA, TypeScript, Angular, AngularJS, Git, PostgreSQL, MongoDB, AI, **Gen AI**, R, Go, Kotlin, Swift, Sass, Vue, SciPy, **AWS**, Cybersecurity, Data Science, Introdução à Programação, Bash, **Rust**, W3.CSS, How To. (A negrito: adições mais recentes, úteis para perceber a direção de crescimento do catálogo.)

Replicar as 45 no MVP não faz sentido — ver recomendação de âmbito na secção 6.

3.2 Padrão estrutural de cada tutorial

Confirmado em HTML, CSS e JavaScript — o mesmo esqueleto repete-se, o que é ótimo porque significa que dá para desenhar **um único template** e aplicá-lo a qualquer linguagem nova:

```
Tutorial (capítulos sequenciais)
  → Examples
  → Exercises
  → Quiz
  → Editor próprio da linguagem
  → "[Linguagem] Website" (mini-projeto: to-do list, slideshow, modal, form validation...)
  → Syllabus / Study Plan
  → Interview Prep
  → Bootcamp (pago – não replicar)
  → Certificate (pago – não replicar)
```

A Reference de cada linguagem tem sub-estrutura própria (ex.: CSS Reference → Selectors, Combinators, Pseudo-classes, Pseudo-elements, At-rules, Functions, Units, Browser Support, Default Values...).

3.3 Editor de código / IDE

Dois editores distintos e tecnicamente diferentes — confirmado pelos próprios URLs:

- **Frontend** (`tryit.asp`) — HTML/CSS/JS, 100% client-side, painel de código + painel de resultado num iframe.
- **Backend** (`trycompiler.asp`) — Python/PHP/Java/C/etc. O próprio nome ("compiler") já indica execução do lado do servidor.

Spaces é o mais próximo que a W3Schools tem de um "IDE embutido": ambiente cloud sem instalação, com Terminal & Log, File Navigator, suporte multi-ficheiro, ambiente

explicitamente descrito como "sandboxed" no marketing deles. Tem dois níveis: **Basic** (só frontend) e **Full-Stack** (acrescenta Python, PHP, Node.js, React, Vue, Django).

3.4 Exercícios, quizzes, certificados, gamificação

Certificados são pagos e emitidos externamente via `campus.w3schools.com`. Existe também uma **League** (XP + ranks) e um mini-jogo chamado **Code Game** ("ajuda o lince a apanhar pinhas") — ou seja, a W3Schools já valida gamificação como parte do produto, mas de forma simples (XP linear, sem adaptação). O que tens desenhado (pomodoro + streaks + spaced repetition + grafo adaptativo) é estruturalmente mais rico do que isto.

3.5 Referência técnica

Cobre cerca de 30 tecnologias, sempre no mesmo formato: sintaxe + descrição + exemplo + tabela de suporte de browser.

3.6 Achado relevante — "kAI"

A W3Schools tem um assistente de IA próprio, o **kAI**, que se apresenta como tutor de IA e ajuda a depurar código, melhorar a estrutura do código e explicar conceitos — mas é cloud e está ligado à conta/subscrição paga.

Isto interessa por dois motivos: confirma que um "buddy" de IA faz sentido dentro deste tipo de produto, e dá-te uma diferenciação clara e fácil de comunicar para o CopyKate — **local, sandboxed, e a treinar a partir do teu próprio estilo de código**, em vez de um genérico tutor cloud. Deixei isto explícito no prompt (ficheiro 2) para o executor não reinventar "um kAI genérico" por engano em vez do CopyKate que já desenhaste.

3.7 Hipótese de arquitetura backend do W3Schools

Sem acesso ao código deles — mas com evidência observável direta, que distingo do que é inferência:

- **Observado diretamente:** os URLs de conteúdo ainda usam `.asp` (ex.: `html/default.asp`, `tryit/trycompiler.asp`) — indica stack legado tipo Classic ASP/ASP.NET para as páginas, consistente com um site online desde 1999. Google Tag Manager está presente no HTML da homepage (stack de ads/analytics — a excluir da tua versão).
- **Inferência razoável, não confirmada:** a execução frontend corre inteiramente no browser (iframe + `srcdoc`, sem round-trip ao servidor) — daí ser gratuita e instantânea mesmo no plano Basic. A execução backend (`trycompiler.asp`, Spaces Full-Stack) tem de correr em sandboxes efémeras do lado do servidor com limites de tempo/recursos — a nomenclatura e o marketing ("sandboxed environment") apontam para isolamento por contentor ou equivalente. O conteúdo de tutoriais/referência é provavelmente servido de forma majoritariamente estática/cacheada, dado o volume e a idade do site.

3.8 O que NÃO replicar

Contas via OAuth (Google/Facebook/GitHub), certificados pagos, subscrição Plus, hosting externo com domínio/SSL, anúncios, Academy/Bootcamps/Newsletter/Community — tudo

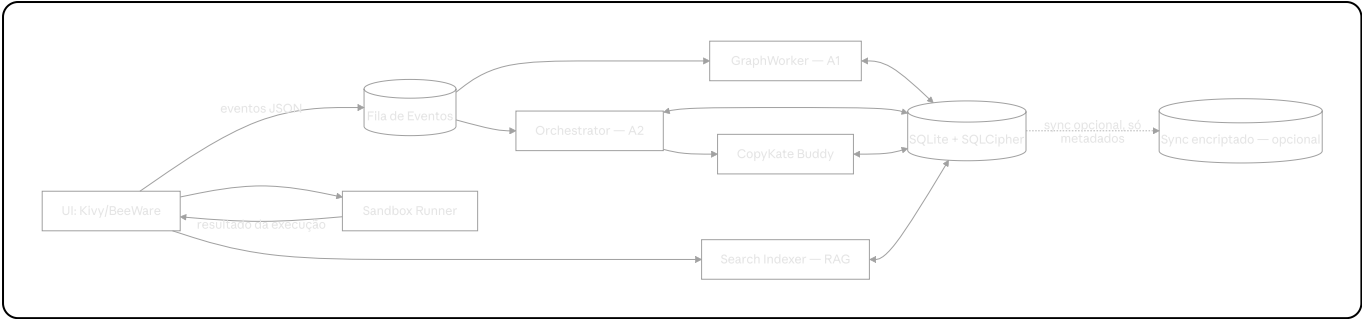
isto depende de infraestrutura cloud e de um modelo de negócio que não se aplica a um produto offline, sem login, de uso pessoal.

4. Mapeamento W3Schools → CopyKate (adaptado)

W3Schools (real)	CopyKate (adaptado)
Tutorials (capítulos por linguagem)	Aprender — mesma lógica de progressão por capítulos, conteúdo 100% original, prática antes da teoria
Try it Yourself — frontend (<code>tryit.asp</code>)	IDE embutido, modo frontend — iframe sandbox local, sem servidor
Try it Yourself — backend (<code>trycompiler.asp</code>)	IDE embutido, modo backend — subprocess/WASM sandboxed, local, sem servidor
Spaces (hosting, projetos persistentes, Terminal & Log)	"Os Meus Projetos" — guardado localmente (SQLite/ficheiros), sem hosting nem domínio
Exercises + Quiz (estáticos)	Exercícios gerados dinamicamente pelo Orchestrator (A2), dificuldade adaptada pelo A1
Certificates (pagos, emitidos externamente)	Emblemas/certificados locais, gerados no dispositivo, sem custo nem verificação externa
League (XP, ranks)	Grafo de progresso (A1) + streaks + spaced repetition — mais granular que XP linear
kAI (assistente cloud genérico)	CopyKate — local, sandboxed, aprende o teu próprio padrão de código
Log in / Sign Up (OAuth)	Perfil local anónimo; export/import manual (JSON)
Where To Start (guia inicial)	Onboarding calibrado pelo A1 (avaliação de nível inicial)
Ads, Plus, Academy, Bootcamps, Newsletter, Community	Fora de âmbito — não aplicável a um produto offline sem monetização

5. Arquitetura — visão rápida

O schema de BD e a especificação completa de cada serviço ficam no ficheiro 2 (é aí que têm de ser tratados com rigor). Aqui fica só o fluxo geral, para orientar:



6. Decisões já assumidas vs. decisões em aberto

Decisão	Estado	Nota
UI framework: Kivy/BeeWare	Assumida (histórico do projeto)	O teu rascunho também aponta React Native/Flutter+Python como alternativa — mantenho Kivy/BeeWare como base por já estar decidido; a alternativa fica registada no prompt como plano B
BD: SQLite + SQLCipher opcional	Assumida	—
Modelos locais: ONNX Runtime Mobile	Assumida	Fallback rule-based se não houver inferência viável no dispositivo
Grafo: NetworkX + D3.js	Assumida	Há espaço para usar métricas como centralidade (para identificar competências "charneira") ou deteção de comunidades (para agrupar tópicos relacionados) no motor A1 — encaixam bem no que já é NetworkX
Sandboxing de código Python em iOS	Em aberto	A Apple restringe execução dinâmica de código/JIT em apps da App Store — correr um subprocess Python arbitrário pode não ser viável nessa plataforma. Ver risco na secção 7.
Linguagens no núcleo do MVP	Em aberto	Recomendo arrancar com HTML/CSS/JS + Python + SQL (o núcleo mais alinhado com o currículo TDAI) e desenhar a arquitetura de forma extensível para as restantes 40 depois
Sincronização opcional entre dispositivos	Em aberto	O teu rascunho já assume "sync opcional encriptado, só metadados" — falta confirmar se isto entra já no MVP ou fica para depois

7. Riscos e pontos de atenção

- Sandboxing seguro de execução de código** é o maior risco técnico do projeto. Mal implementado, um exercício "corre o teu código" vira uma forma de correr código arbitrário sem limites. Regras mínimas: sem acesso a rede a partir do sandbox, limites

de CPU/memória/tempo, sem acesso ao sistema de ficheiros fora de uma pasta temporária.

- **iOS não permite execução dinâmica de código nativa** da forma como o farias em desktop. Isto tem de entrar na arquitetura cedo (ex.: Pyodide/WASM num webview local em vez de subprocess), ou o IDE completo em iOS fica limitado, no MVP, a HTML/CSS/JS.
- **Direitos de autor:** a estrutura pedagógica (capítulos, exercícios, referência) não é protegida, mas o texto exato da W3Schools é. Todo o conteúdo deve ser escrito de raiz, inspirado na estrutura observada, nunca copiado.
- **Tamanho dos modelos locais** (ONNX quantizado) vs. hardware típico de telemóvel — vale a pena fixar um limite (ex.: <200MB por modelo) como critério de aceitação já na Fase 1.
- **RAG local em dispositivo modesto** — FAISS/HNSW com muitos chunks pode pesar; validar com um dataset pequeno antes de escalar o conteúdo.

8. Sitemap de arranque (esqueleto)

Ponto de partida para a Fase 1 elaborar por completo — não é o sitemap final:

```
Início (Dashboard local)
├─ Aprender
│   ├─ HTML / CSS / JavaScript / Python / SQL [núcleo do MVP]
│   └─ [extensível para as restantes linguagens]
│       └─ por track: Capítulos → Exemplos → Exercícios → Quiz → Projeto
prático → Emblema local
├─ IDE / Playground
│   ├─ Editor Frontend (HTML/CSS/JS, sandbox iframe)
│   ├─ Editor Backend (Python/SQL, sandbox subprocess ou WASM)
│   └─ Os Meus Projetos (guardados localmente)
├─ Referência Técnica (por tecnologia)
├─ O Meu Progresso
│   ├─ Grafo de competências (A1)
│   ├─ Estatísticas (tempo, streaks, precisão)
│   └─ Emblemas / certificados locais
├─ CopyKate
│   ├─ Desafios personalizados
│   ├─ Histórico de sessões
│   └─ Privacidade – o que é usado localmente
├─ Foco (Pomodoro)
├─ Pesquisa (RAG local)
├─ Definições
│   ├─ Perfil local (exportar / importar)
│   ├─ Atualização de conteúdo offline (opcional, com registo)
│   └─ Privacidade e dados
└─ Ajuda / Sobre
```

9. Como usar o prompt em anexo

O ficheiro `prompt-copykate-w3schools.md` está pronto a colar num LLM ou swarm à parte, ou a correr aqui mesmo — nesse caso, dado o tamanho do output esperado, faz sentido corrê-lo por secções em vez de tudo de uma vez. É autocontido: não depende deste plano para fazer sentido.

10. Próximos passos imediatos

- ☐ Confirmar as decisões em aberto da secção 6 (linguagens do núcleo, sandboxing em iOS, sync opcional)
- ☐ Rever o prompt (ficheiro 2) e ajustar o que quiseres antes de o correr
- ☐ Correr o prompt — aqui por fases, ou externamente
- ☐ Validar o sitemap/especificação gerados face a este plano
- ☐ Arrancar Sprint 1